Distributed Systems

(4th edition, version 01)

Chapter 08: Fault Tolerance

Dependability

Basics

A component provides services to clients. To provide services, the component may require the services from other components ⇒ a component may depend on some other component.

Specifically

A component C depends on C^* if the correctness of C's behavior depends on the correctness of C^*'s behavior. (Components are processes or channels.)

Requirements related to dependability

Requirement	Description
Availability	Readiness for usage
Reliability	Continuity of service delivery
Safety	Very low probability of catastrophes
Maintainability	How easy can a failed system be repaired

Reliability versus availability

Reliability $R(t)$ of component C

Conditional probability that C has been functioning correctly during $[0, t)$ given C was functioning correctly at time $T = 0$.

Traditional metrics

- Mean Time To Failure (*MTTF*): The average time until a component fails.
- Mean Time To Repair (*MTTR*): The average time needed to repair a component.
- Mean Time Between Failures (*MTBF*): Simply *MTTF* + *MTTR*.

Reliability versus availability

Availability $A(t)$ of component C

Average fraction of time that C has been up-and-running in interval $[0, t)$.

- Long-term availability A: $A(\infty)$
- Note: $A = \frac{MTTF}{MTBF} = \frac{MTTF}{MTTF+MTTR}$

Observation

Reliability and availability make sense only if we have an accurate notion of what a failure actually is.

Terminology

Failure, error, fault

Term	Description	Example
Failure	A component is not living up to its specifications	Crashed program
Error	Part of a component that can lead to a failure	Programming bug
Fault	Cause of an error	Sloppy programmer

Terminology

Handling faults

Term	Description	Example
Fault prevention	Prevent the occurrence of a fault	Don't hire sloppy programmers
Fault tolerance	Build a component such that it can mask the occurrence of a fault	Build each component by two independent programmers
Fault removal	Reduce the presence, number, or seriousness of a fault	Get rid of sloppy programmers
Fault forecasting	Estimate current presence, future incidence, and consequences of faults	Estimate how a recruiter is doing when it comes to hiring sloppy programmers

Failure models

Types of failures

Type	Description of server's behavior
Crash failure	Halts, but is working correctly until it halts
Omission failure	Fails to respond to incoming requests
Receive omission	Fails to receive incoming messages
Send omission	Fails to send messages
Timing failure	Response lies outside a specified time interval
Response failure	Response is incorrect
Value failure	The value of the response is wrong
State-transition failure	Deviates from the correct flow of control
Arbitrary failure	May produce arbitrary responses at arbitrary times

Dependability versus security

Omission versus commission

Arbitrary failures are sometimes qualified as malicious. It is better to make the following distinction:

- Omission failures: a component fails to take an action that it should have taken
- Commission failure: a component takes an action that it should not have taken

Observation

Note that deliberate failures, be they omission or commission failures, are typically security problems. Distinguishing between deliberate failures and unintentional ones is, in general, impossible.

Halting failures

Scenario

C no longer perceives any activity from C^*— a halting failure? Distinguishing between a crash or omission/timing failure may be impossible.

Asynchronous versus synchronous systems

- Asynchronous system: no assumptions about process execution speeds or message delivery times → cannot reliably detect crash failures.
- Synchronous system: process execution speeds and message delivery times are bounded → we can reliably detect omission and timing failures.
- In practice we have partially synchronous systems: most of the time, we can assume the system to be synchronous, yet there is no bound on the time that a system is asynchronous → can normally reliably detect crash failures.

Halting failures

Assumptions we can make

Halting type	Description
Fail-stop	Crash failures, but reliably detectable
Fail-noisy	Crash failures, eventually reliably detectable
Fail-silent	Omission or crash failures: clients cannot tell what went wrong
Fail-safe	Arbitrary, yet benign failures (i.e., they cannot do any harm)
Fail-arbitrary	Arbitrary, with malicious failures

Redundancy for failure masking

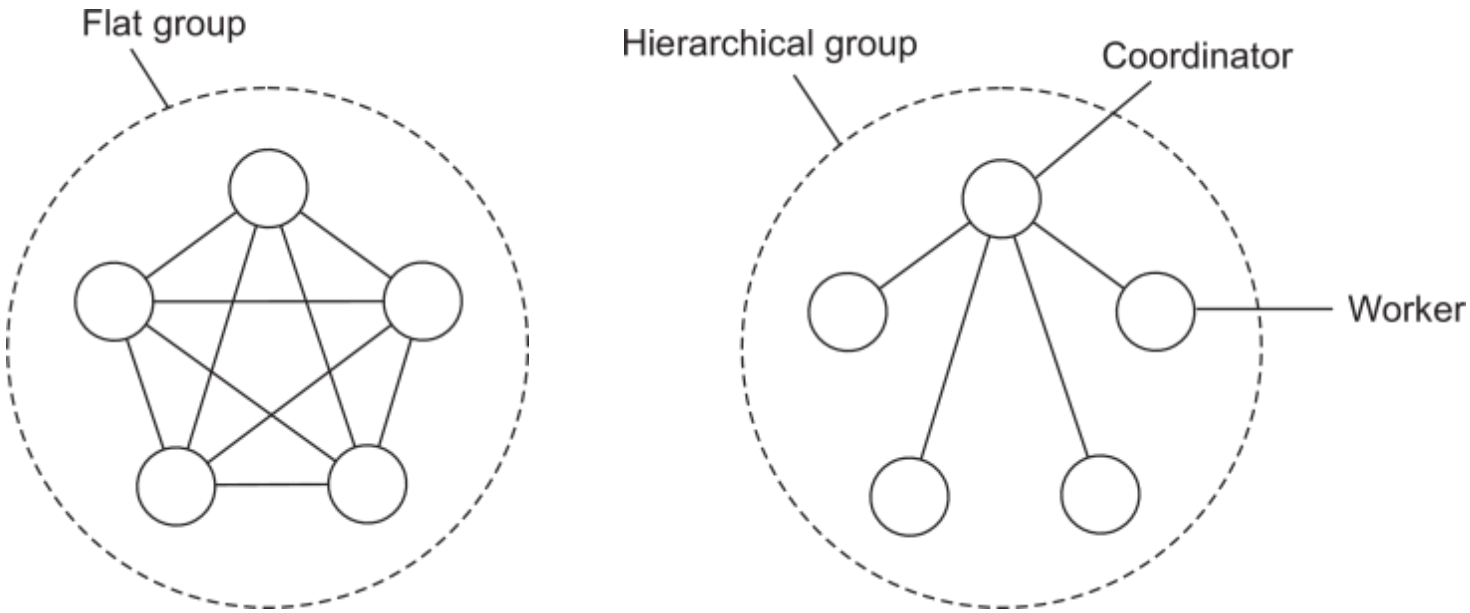
Types of redundancy

- Information redundancy: Add extra bits to data units so that errors can recovered when bits are garbled.
- Time redundancy: Design a system such that an action can be performed again if anything went wrong. Typically used when faults are transient or intermittent.
- Physical redundancy: add equipment or processes in order to allow one or more components to fail. This type is extensively used in distributed systems.

Process resilience

Basic idea

Protect against malfunctioning processes through process replication, organizing multiple processes into a process group. Distinguish between flat groups and hierarchical groups.



Groups and failure masking

k-fault tolerant group

When a group can mask any *k* concurrent member failures (*k* is called degree of fault tolerance).

How large does a *k*-fault tolerant group need to be?

- With halting failures (crash/omission/timing failures): we need a total of *k* + 1 members as no member will produce an incorrect result, so the result of one member is good enough.
- With arbitrary failures: we need 2*k* + 1 members so that the correct result can be obtained through a majority vote.

Important assumptions

- All members are identical
- All members process commands in the same order

Result: We can now be sure that all processes do exactly the same thing.

Consensus

Prerequisite

In a fault-tolerant process group, each nonfaulty process executes the same commands, and in the same order, as every other nonfaulty process.

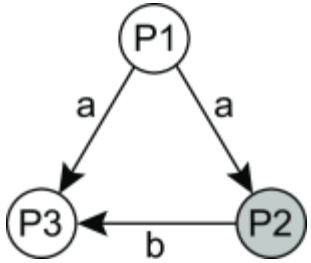
Reformulation

Nonfaulty group members need to reach consensus on which command to execute next.

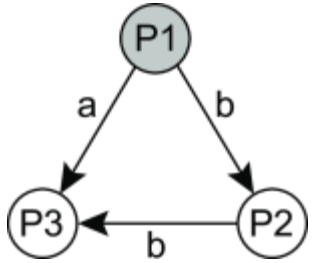
Consensus under arbitrary failure semantics

Essence

We consider process groups in which communication between process is inconsistent.



Improper forwarding



Different messages

Consensus under arbitrary failure semantics

System model

- We consider a primary P and $n-1$ backups $B_1, \ldots, B_{n-1}$.
- A client sends $v \in \{T, F\}$ to P
- Messages may be lost, but this can be detected.
- Messages cannot be corrupted beyond detection.
- A receiver of a message can reliably detect its sender.

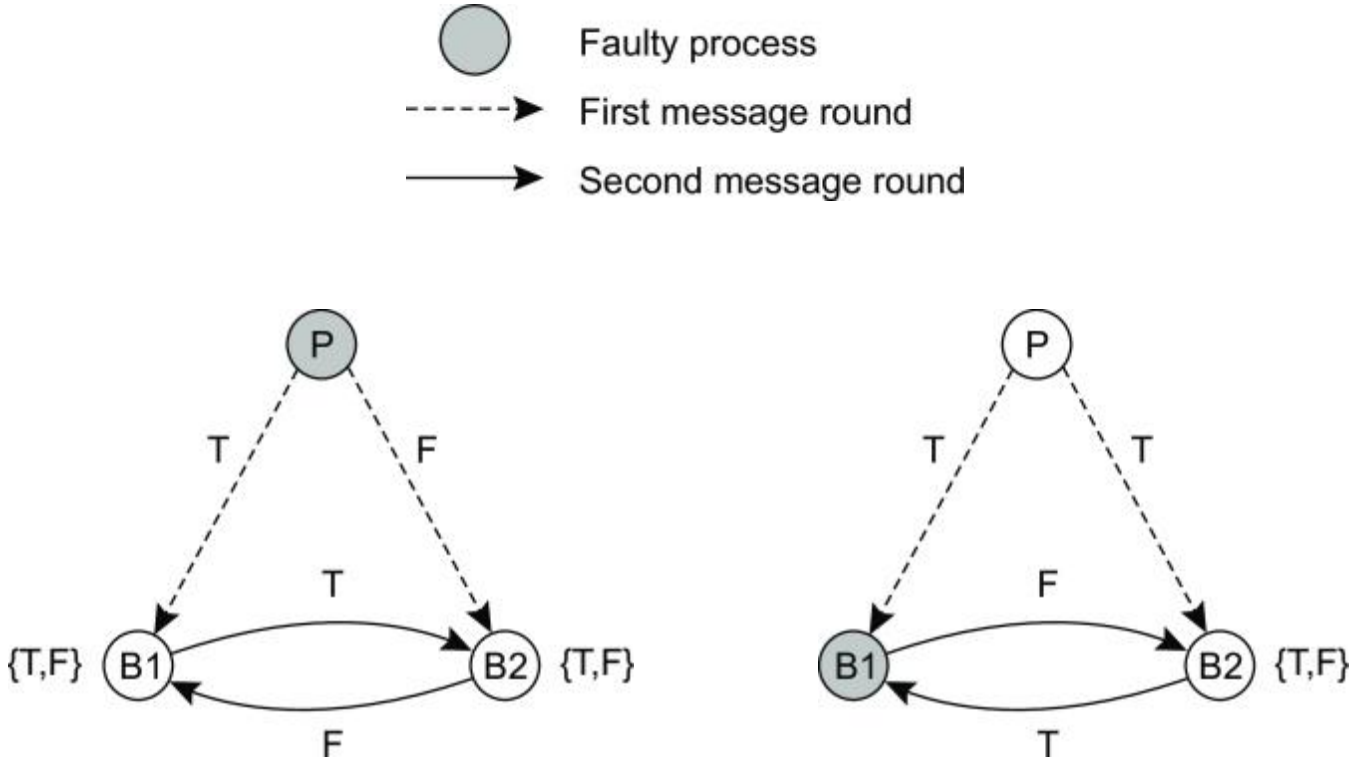
Byzantine agreement: requirements

BA1: Every nonfaulty backup process stores the same value.
BA2: If the primary is nonfaulty then every nonfaulty backup process stores exactly what the primary had sent.

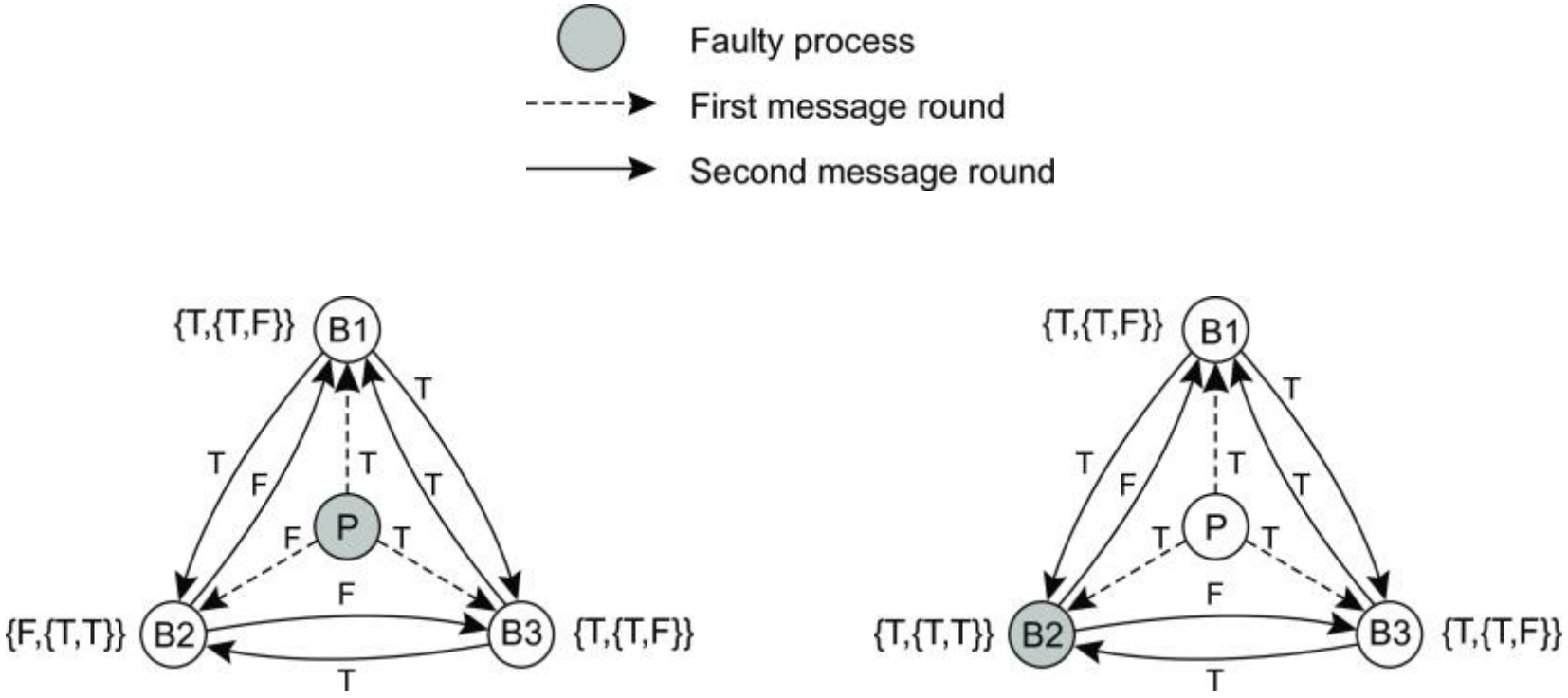
Observation

- Primary faulty $\Rightarrow$ BA1 says that backups may store the same, but different (and thus wrong) value than originally sent by the client.
- Primary not faulty $\Rightarrow$ satisfying BA2 implies that BA1 is satisfied.

Why having **3k** processes is not enough



Why having **3k** + **1** processes is enough



Consistency, availability, and partitioning

CAP theorem

Any networked system providing shared data can provide only two of the following three properties:

- C: consistency, by which a shared and replicated data item appears as a single, up-to-date copy
- A: availability, by which updates will always be eventually executed
- P: Tolerant to the partitioning of process group.

Conclusion

In a network subject to communication failures, it is impossible to realize an atomic read/write shared memory that guarantees a response to every request.

Failure detection

Issue

How can we reliably detect that a process has actually crashed?

General model

- Each process is equipped with a failure detection module
- A process *P* probes another process *Q* for a reaction
- If *Q* reacts: *Q* is considered to be alive (by *P*)
- If *Q* does not react with *t* time units: *Q* is suspected to have crashed

Observation for a synchronous system

a suspected crash ≡ a known crash

Practical failure detection

Implementation

- If *P* did not receive heartbeat from *Q* within time *t*: *P* suspects *Q*.
- If *Q* later sends a message (which is received by *P*):
  - *P* stops suspecting *Q*
  - *P* increases the timeout value *t*
- Note: if *Q* did crash, *P* will keep suspecting *Q*.

Reliable remote procedure calls

What can go wrong?

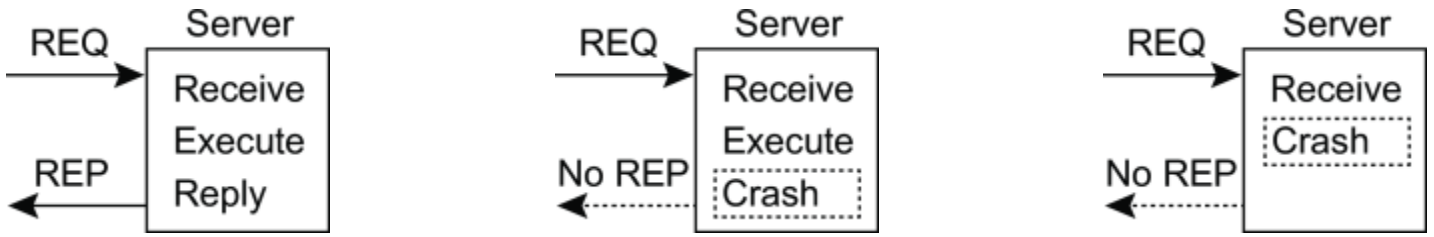
1. The client is unable to locate the server.
2. The request message from the client to the server is lost.
3. The server crashes after receiving a request.
4. The reply message from the server to the client is lost.
5. The client crashes after sending a request.

Two “easy” solutions

1: (cannot locate server): just report back to client

2: (request was lost): just resend message

Reliable RPC: server crash



(a) (b) (c)

Problem

Where (a) is the normal case, situations (b) and (c) require different solutions. However, we don't know what happened. Two approaches:

- At-least-once-semantics: The server guarantees it will carry out an operation at least once, no matter what.
- At-most-once-semantics: The server guarantees it will carry out an operation at most once.

Reliable RPC: lost reply messages

The real issue

What the client notices, is that it is not getting an answer. However, it cannot decide whether this is caused by a lost request, a crashed server, or a lost response.

Partial solution

Design the server such that its operations are idempotent: repeating the same operation is the same as carrying it out exactly once:

- pure read operations
- strict overwrite operations

Many operations are inherently nonidempotent, such as many banking transactions.

Reliable RPC: client crash

Problem

The server is doing work and holding resources for nothing (called doing an orphan computation).

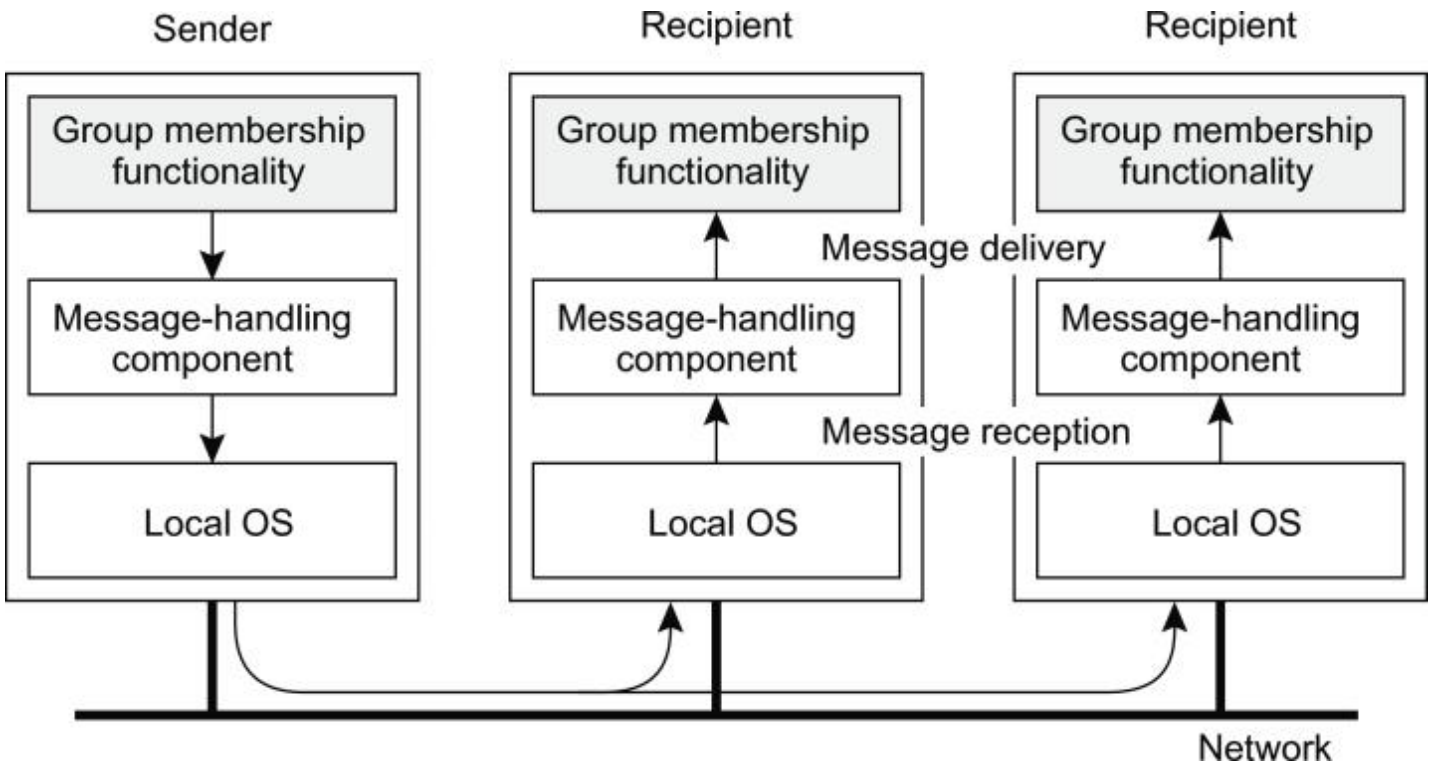
Solution

- Orphan is killed (or rolled back) by the client when it recovers
- Client broadcasts new epoch number when recovering ⇒ server kills client's orphans
- Require computations to complete in a T time units. Old ones are simply removed.

Simple reliable group communication

Intuition

A message sent to a process group **G** should be delivered to each member of **G**. Important: make distinction between receiving and delivering messages.



Recovery: Background

Essence

When a failure occurs, we need to bring the system into an error-free state:

- Forward error recovery: Find a new state from which the system can continue operation
- Backward error recovery: Bring the system back into a previous error-free state

Practice

Use backward error recovery, requiring that we establish recovery points

Observation

Recovery in distributed systems is complicated by the fact that processes need to cooperate in identifying a consistent state from where to recover

Message logging

Alternative

Instead of taking an (expensive) checkpoint, try to replay your (communication) behavior from the most recent checkpoint ⇒ store messages in a log.

Assumption

We assume a piecewise deterministic execution model:

- The execution of each process can be considered as a sequence of state intervals
- Each state interval starts with a nondeterministic event (e.g., message receipt)
- Execution in a state interval is deterministic

Conclusion

If we record nondeterministic events (to replay them later), we obtain a deterministic execution model that will allow us to do a complete replay.